

システム科学科 情報処理演習テキスト 9

－ グラフィックス －

原案:佐藤宏介 修正: 升谷 保博, 庄野 逸, 鳩野逸生, 橋本守, 才脇直樹, 田中宏喜, 日浦慎作

注意 1: グラフィックスを使用する場合は compile 時にオプション -lhg をつけること

gcc example91.c -o example91 -lhg

注意 2: グラフィックスを使用する場合は “handy.h” を include しておくこと

グラフィックスを使用する時の構造

ウィンドウの用意: HgWOpen(ウィンドウの左座標, 下座標, 幅, 高さ);

図形を保存するための準備: HgWInitEPS(w, "ファイル名");

ウィンドウを閉じる: HgWClose(w);

ポイント 1: グラフィックス関数を使用する場合は, テンプレートのような構造を取ればよい

テンプレート 9_1

```
/* 実行したらリターンキーを何回か入力してみよ */
#include <stdio.h>
#include <handy.h>

int main( void )
{
    int w;

    w = HgWOpen( 200.0, 150.0, 500.0, 400.0 );
    HgWInitEPS( w, "file.eps" ); /* 絵を file.eps に保存する用意 */
    /* 次の行からお絵書きのプログラムを書く */

    /* ここまで絵のプログラム */
    HgWClose( w );
    return 0;
}
```

example9_1.c

```
/* 実行したらリターンキーを何回か入力してみよ */
#include <stdio.h>
#include <handy.h>

int main( void )
{
    int w;

    w = HgWOpen( 200.0, 150.0, 500.0, 400.0 ); /* Window を開きます*/
    HgWInitEPS(w, "example91.eps"); /* eps ファイルに保存します*/
    /* 何の絵かわかる人います ? */

    HgWWidth(w, 2.0);
    HgWLine( w, 250.0, 200.0, 200.0, 150.0);
    getchar();
    HgWCircle( w, 250.0, 200.0, 100.0);
    getchar();
    HgWCircleFill( w, 250.0, 200.0, 50.0);
    getchar();
    HgWBox( w, 200.0, 20.0, 50.0, 80.0);
    getchar();
    HgWBoxFill( w, 250.0, 20.0, 50.0, 80.0);
    getchar();
    HgWText( w, 350.0, 50.0, "Kitaro !!");
    getchar();

    HgWClose( w ); /* Window を閉じる */
    return 0;
}
```

ポイント 2: getchar() は, 本来 (キーボードから) 1 文字読み込む関数だが, ここでは実行を一旦停止するために使っている

直線: HgWLine(w, 始点 x 座標, 始点 y 座標, 終点 x 座標, 終点 y 座標);

線の太さ: HgWWidth(w, 線の太さ t);

円: HgWCircle(w, 円の中心の x 座標, 円の中心の y 座標, 半径 r);

塗りつぶし円: HgWCircleFill(w, 円の中心の x 座標, 円の中心の y 座標, 半径 r);

四角形: HgWBox(w, 左下隅の x 座標, 左下隅の y 座標, 幅 w, 高さ h);

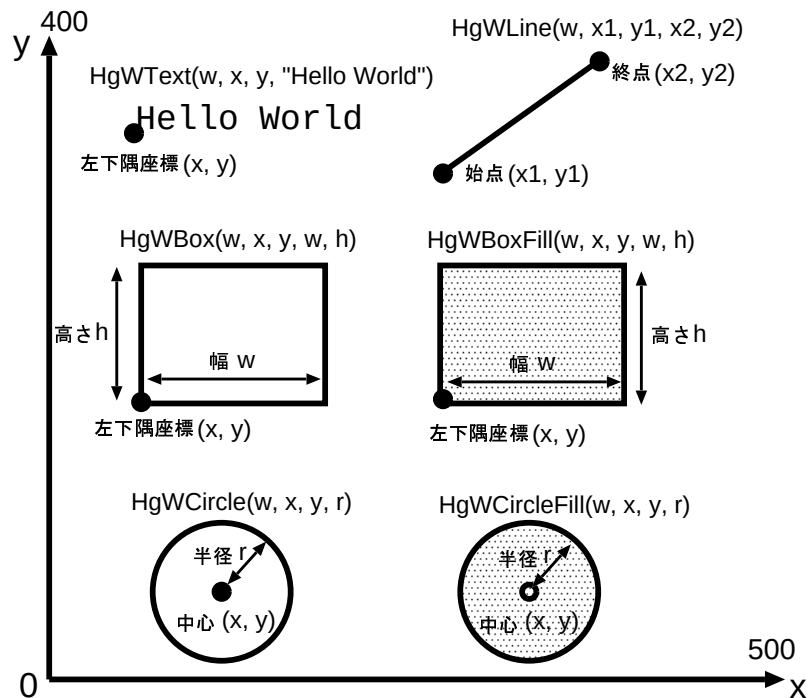
塗りつぶし四角形: HgWBoxFill(w, 左下隅の x 座標, 左下隅の y 座標, 幅 w, 高さ h);

文字列: HgWText(w, 左下隅の x 座標, 左下隅の y 座標, 文字列);

注意 2: HandyGraphics の引数は実数型なので、数値には小数点をつけたほうが良い (0.0, 10.0, 123.4 など)

注意 3: 「文字列」とは文字のならびの両側を「」で挟んだものである。「」 (double quotation mark) は日本語入力をオフにして Shift をおしながら 2 を押すことで入力する。

注意 4: ファイル名には英数字をつかうこと



情報処理演習の Web ページにも例が乗っているので参照のこと
http://sml.me.es.osaka-u.ac.jp/jse/hg_examples/

example9_2.c

```

/*
 * For ループを使って、白黒円の市松模様を描く
 */
#include <stdio.h>
#include <handy.h>
#define IXMAX 10
#define IYMAX 7
#define RADIUS 25.0

int main(void)
{
    int w;
    int ix, iy;
    double xc, yc;

    w = HgWOpen(100.0, 100.0, 2*RADIUS*IXMAX, 2*RADIUS*IYMAX); /* ウィンドウの作成 */
    HgWInitEPS(w, "example92.eps"); /* eps ファイルに保存 */
    HgWWidth(w, 2.0); /* 線幅を 2.0 に設定 */

    for ( ix = 0; ix < IXMAX; ix++ ) {
        xc = RADIUS + 2*RADIUS*ix;
        for ( iy = 0; iy < IYMAX; iy++ ) {
            yc = RADIUS + 2*RADIUS*iy;
            if ( (ix+iy)%2 == 0 ) {
                HgWCircle(w, xc, yc, RADIUS); /* 格子状に○と●を並べる */
            } else {
                HgWCircleFill(w, xc, yc, RADIUS); /* 市松模様になるように○と●を切替え */
            }
        }
    }

    getchar();
    HgWClose(w); /* ウィンドウを閉じる */
    return 0;
}

```

ポイント 3: 細かな直線をつなげることで、曲線を描くことができる。

example9_3.c

```
/*
 * sprintf() を用いたプログラム例
 */
#include <stdio.h>
#include <handy.h>
#include <stdlib.h>      /* rand(), srand(), RAND_MAX のために必要 */
#include <time.h>        /* time(), のために必要 */
#define MAXTIME 10      /* カウントする秒数 */
#define MINTIME 5

int main( void )
{
    int w, color;
    int i, mtime, ctime, ttime, stime;
    char buf[50];

    w = HgWOpen(100.0, 100.0, 300.0, 100.0 );
    ctime = 0;
    mtime = MAXTIME;
    stime = time( NULL );

    while( 1 ){
        if( ctime >= mtime ){
            break;
        }
        ttime = time( NULL ) - stime;
        if( ttime > ctime ){
            ctime = ttime;
            HgWClear( w );
            sprintf( buf, "Last %02d second to vanish!!", mtime - ctime );

            /* 残り時間が MINTIME を切ったら色を変えてみる */
            if( mtime - ctime <= MINTIME ){
                color = HgRGB( 1.0, 0.0, 0.0 );      /* 赤色 */
                HgColor( color );
            }
            HgWText( w, 25.0, 40.0, buf );

        }
        HgWClose(w);
        return 0;
    }
}
```

ポイント 4: sprintf() 関数を用いることにより、画面出力を文字列中に書き込むことが可能。書式指定文字列は、printf() 関数をまったく同じものが使用できる

ポイント 5: sprintf() 関数は、Handygraphic の HgWText() 関数などで文字を描画するときに便利である。

example9_4.c

```
/*
 * 細かな直線をつなげて描くことで、曲線を描く例
 */
#include <stdio.h>
#include <math.h>
#include <handy.h>
#define PI 3.141592653589793
#define WIDTH 500.0      /* Window の幅*/
#define HEIGHT 500.0     /* Window の高さ*/

int main(void)
{
    int w, i, imax;
    double x0, y0, r, x, y, t, xPrev, yPrev;

    w = HgWOpen( 100.0, 100.0, WIDTH, HEIGHT );
    HgWInitEPS(w, "example93.eps");      /* eps ファイルで保存 */

    x0=WIDTH/2.0;
    y0=HEIGHT/2.0;
    r=100.0;
    imax=32;
    xPrev = x0 + r*cos(0.0);
    yPrev = y0 + r*sin(0.0);

    for ( i = 1; i <= imax; i++ ) {
        t = 2.0*PI/imax*i;
        x = x0 + r*cos(t);
        y = y0 + r*sin(t);
        HgWLine(w, x, y, xPrev, yPrev);
        xPrev = x;
        yPrev = y;
    }
    getchar();
    HgWClose(w);
    return 0;
}
```

```

/*
 * 細かな直線を連続して描くことで、曲線を描く例.
 * リサージュ曲線を描く.
 *  $x(t) = x_0 + r \cos(a \cdot t)$ 
 *  $y(t) = y_0 + r \sin(b \cdot t)$ 
 * 中心 (x0, y0) がウィンドウ外ならば描かない.
 */
#include <stdio.h>
#include <math.h>
#include <handy.h>

#define PI      3.14159265358979323846      /* 円周率 */
#define XMIN    0.0                        /* x 座標の最小値 */
#define XMAX    500.0                      /* x 座標の最大値 */
#define YMIN    0.0                        /* y 座標の最小値 */
#define YMAX    400.0                      /* y 座標の最大値 */
#define GRID    50.0                      /* 格子の幅 */
#define TBEGIN  0.0                        /* 媒介数変 t の開始値 (rad) */
#define TEND    (2.0*PI)                  /* 媒介数変 t の終了値 (rad) */

int main(void)
{
    int      w;
    double   x0, y0, r, a, b, x, y, xPrev, yPrev, dt, t;
    int      i, imax;

    w = HgWOpen(100.0, 100.0, XMAX, YMAX);      /* ウィンドウの作成 */
    HgWInitEPS(w, "example94.eps");              /* eps ファイルに保存 */

    for ( i = 1; i < (YMAX-YMIN)/GRID; i++ ) {   /* x 軸に平行な格子線 */
        y = YMIN + GRID*i;
        HgWLine(w, XMIN, y, XMAX, y);
    }

    for ( i = 1; i < (XMAX-XMIN)/GRID; i++ ) {   /* y 軸に平行な格子線 */
        x = XMIN + GRID*i;
        HgWLine(w, x, YMIN, x, YMAX);
    }

    /* パラメータの入力 */
    printf("中心の x 座標を入力して下さい→ "); scanf("%lf", &x0);
    printf("中心の y 座標を入力して下さい→ "); scanf("%lf", &y0);
    printf("大きさを入力して下さい      → "); scanf("%lf", &r);
    printf("a を入力して下さい          → "); scanf("%lf", &a);
    printf("b を入力して下さい          → "); scanf("%lf", &b);
    printf("分割数を入力して下さい      → "); scanf("%d", &imax);

    /* 中心座標のチェック */
    if ( x0 > XMAX || x0 < XMIN || y0 > YMAX || y0 < YMIN ) {
        printf("中心座標 (%6.1f, %6.1f) はウィンドウの範囲外です. \n", x0, y0);
        printf("プログラムを終了します. \n");
        exit(1);
    }

    /* 細かい直線の連続でリサージュ曲線を描く */
    xPrev = x0 + r*cos(a*TBEGIN);
    yPrev = y0 + r*sin(b*TBEGIN);
    dt = (TEND-TBEGIN)/imax;

    HgWWidth(w, 2.0);      /* 線幅を 2.0 に設定 */

    for ( i = 1; i <= imax; i++ ) {
        t = TBEGIN + dt*i;
        x = x0 + r*cos(a*t);
        y = y0 + r*sin(b*t);
        HgWLine(w, x, y, xPrev, yPrev);
        xPrev = x;
        yPrev = y;
    }

    HgWText(w, 10.0, 10.0, "ks7999kk Kiso Kotaro"); /* 名前 */

    getchar();

    HgWClose(w);      /* ウィンドウを閉じる */
    return 0;
}

```

● 多角形の描画

HgWPolygon(w, 頂点数, 頂点の x 座標配列, 頂点の y 座標配列);

頂点を示した配列を渡すことで, 頂点を順に結んだ図形を描いてくれる. 詳しくは HandyGraphic マニュアル (http://sml.me.es.osaka-u.ac.jp/jse/hg_examples/) を参照.

また使用例は example96.c, example97.c を参考にすれば良い

example9_6.c

```

/*
 * HgWPolygon を活用して図形を描く.
 * HgWPolygon は 配列を引数として渡される.
 */
#include <stdio.h>
#include <math.h> /* 数学関数 cos 等を使用する時には必要 */
#include <handy.h> /* Handy Graphics を使用する時には必要 */
#define PI 3.14159265358979323846 /* macro 定義を使って円周率を定義 */
#define NUM 5
/* 関数のプロトタイプ宣言 */
void star(int n, int m, double x0, double y0, double r, double ax[], double ay[]);

int main(void)
{
    int w;
    double x[NUM];
    double y[NUM];

    w=HgWOpen(100.0, 100.0, 500.0, 500.0); /* window を開きます */
    HgWInitEPS(w, "example95.eps"); /* eps ファイルで保存 */

    star(NUM, 2, 250.0, 250.0, 150.0, x, y);
    HgWPolygon(w, NUM, x, y);

    getchar(); /* 何かキーボード入力されるのを待つ */
    HgWClose(w); /* window を閉じる */
    return 0;
}

/*
 * 一筆描きの星形の x,y 座標を計算する
 * n: 円周上に取り点数 m: 点を取る間隔
 * x0, y0: 中心座標 r: 半径
 * ax, ay: x 座標と y 座標を格納する配列 (出力)
 */
void star(int n, int m, double x0, double y0, double r, double ax[], double ay[])
{
    int i;
    double dt;

    dt = (2*PI/n)*m;
    for ( i = 0; i < n; i++ ) {
        ax[i] = x0 + r*cos(dt*i);
        ay[i] = y0 + r*sin(dt*i);
    }
}

```

```

/*
 * 配列を引数として渡される関数の例.
 * GrPolygon を活用して一筆描き星形をたくさん描く
 * 配列を関数に渡して, GrPolygon によって描く図形を作成.
 */
#include <stdio.h>
#include <math.h>
#include <handy.h>

#define PI 3.14159265358979323846 /* 円周率 */
#define PMAX 1000
#define RADIUS 50.0
#define NSTAR 10

/* 関数のプロトタイプ宣言 */
void star(int n, int m, double x0, double y0, double r, double ax[], double ay[]);
void translate(double x, double y, int n, double ax[], double ay[]);

int main(void)
{
    int w;
    int i, n, m;
    static double x[PMAX], y[PMAX];

    w = HgWOpen(100.0, 100.0, 2*RADIUS*NSTAR, 2*RADIUS); /* ウィンドウの作成 */
    HgWInitEPS(w, "example76.eps"); /* eps ファイルに保存 */
    HgWWidth(w, 3.0); /* 線幅の指定 */

    for (;;) {
        /* パラメータの入力 */
        printf("円周上の n 個の点を m 個間隔で結びます (不適切な値を入力すると, 終了します). \n");
        printf("n → "); scanf("%d", &n);
        printf("m → "); scanf("%d", &m);
        if ( n < 5 || m < 2 || m >= n || n%m == 0 ) {
            printf("パラメータが不適です. \n");
            break;
        }
        HgWClear(w); /* ウィンドウの消去 */
        star(n, m, RADIUS, RADIUS, RADIUS, x, y); /* 最初の星形の座標を計算 */
        for ( i = 0; i < NSTAR; i++ ) { /* x 座標を変化させながら描く */
            HgWPolygon(w, n, x, y);
            translate(2*RADIUS, 0.0, n, x, y);
        }
        HgWClose(w);
        printf("プログラムを終了します. \n");
        return 0;
    }
}

/*
 * 一筆描きの星形の x,y 座標を計算する
 * n: 円周上に取る点数
 * x0, y0: 中心座標
 * ax, ay: x 座標と y 座標を格納する配列 (出力)
 * m: 点を取る間隔
 * r: 半径
 */
void star(int n, int m, double x0, double y0, double r, double ax[], double ay[])
{
    int i;
    double dt;

    dt = (2*PI/n)*m;
    for ( i = 0; i < n; i++ ) {
        ax[i] = x0 + r*cos(dt*i);
        ay[i] = y0 + r*sin(dt*i);
    }
}

/*
 * x, y 座標の入った配列の並進変換
 * x, y: x, y の並進量
 * ax, ay: x 座標と y 座標を格納する配列 (入力/出力)
 * n: データ数
 */
void translate(double x, double y, int n, double ax[], double ay[])
{
    int i;

    for ( i = 0; i < n; i++ ) {
        ax[i] += x;
        ay[i] += y;
    }
}

```